



Introduction to NLP

AIAA 4051 Lecture 2

Instructor: Sihong Xie

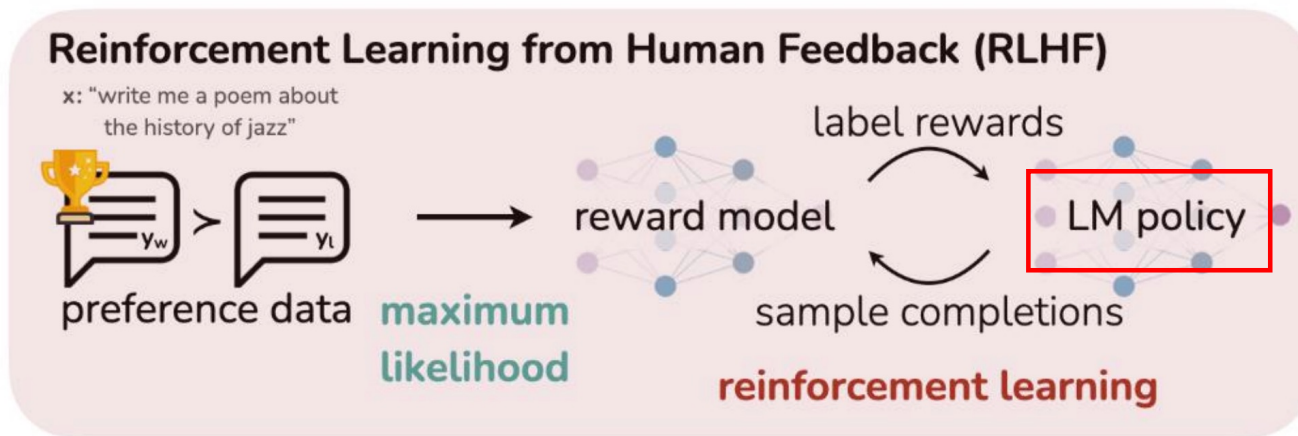
Email: sihongxie@hkust-gz.edu.cn

AI Thrust, HKUST(GZ)

Last Lecture: RLHF

RHLF = Preference data + Reward Model (RM) + Reinforcement Learning (RL)

The model is optimized using reinforcement learning **algorithms (typically PPO)** based on the RM score.



After the reward model is trained using Bradley-Terry model on preference data, the reward model is frozen during reinforcement learning for the LM policy (the target LLM), which is pre-trained before.

| This Lecture: PPO

We will first introduce typical policy gradient in reinforcement learning.

Then we explain:

1. Natural Policy Gradient (NPG)
2. Trust Region Policy Optimization (TRPO)
3. Proximal Policy Optimization (PPO)

Policy learning from delayed rewards

A policy is the brain of an agent that determines the next action to take in each situation. It is typically a neural network to perceive / represent complex situations.

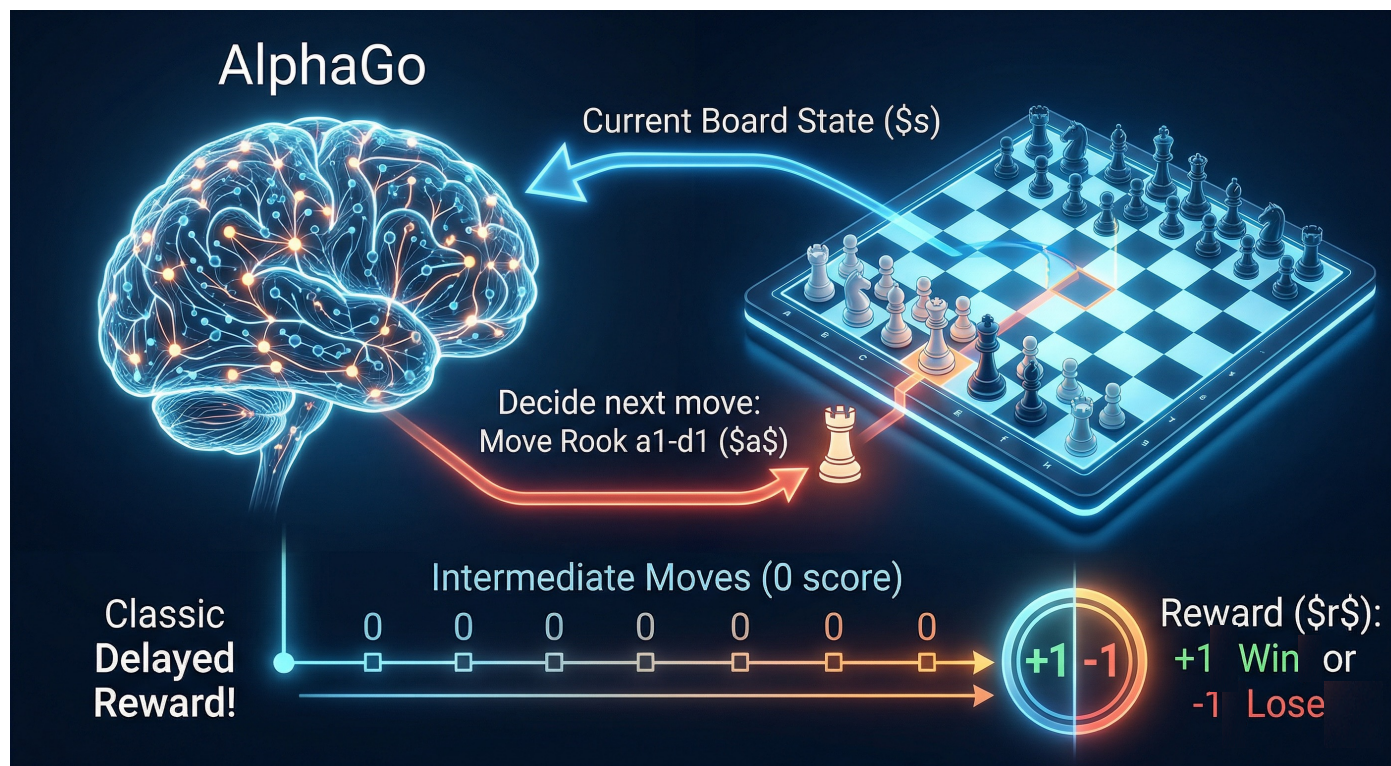


Image source: Gemini

Similar to play the Go game, for language generation, the reward is **delayed**, only known when the full generation is done.

Example

Q: what's the capital of China?

A: the capital of China is Beijing.

Beijing is generated to earn a reward.

How to train a policy

Compute the policy gradient and update the policy parameters.

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s}) \nabla_{\theta} \log p_{\theta}(\hat{s})] \approx \frac{1}{m} \sum_{i=1}^m R(s_i) \nabla_{\theta} \log p_{\theta}(s_i)$$

$$V(\theta_k) = \mathbb{E}_{\hat{s} \sim p_{\theta_k}(s)} [R(\hat{s})]$$

θ_{k+1} : New Brain (Param)

θ_k : Current Brain (Param)

α_k : Learning Rate

$$\theta_{k+1} = \theta_k + \alpha_k \nabla_{\theta} V(\theta_k)$$

$\nabla_{\theta} V(\theta_k)$: Policy Gradient

How to choose the learning rate (step)?

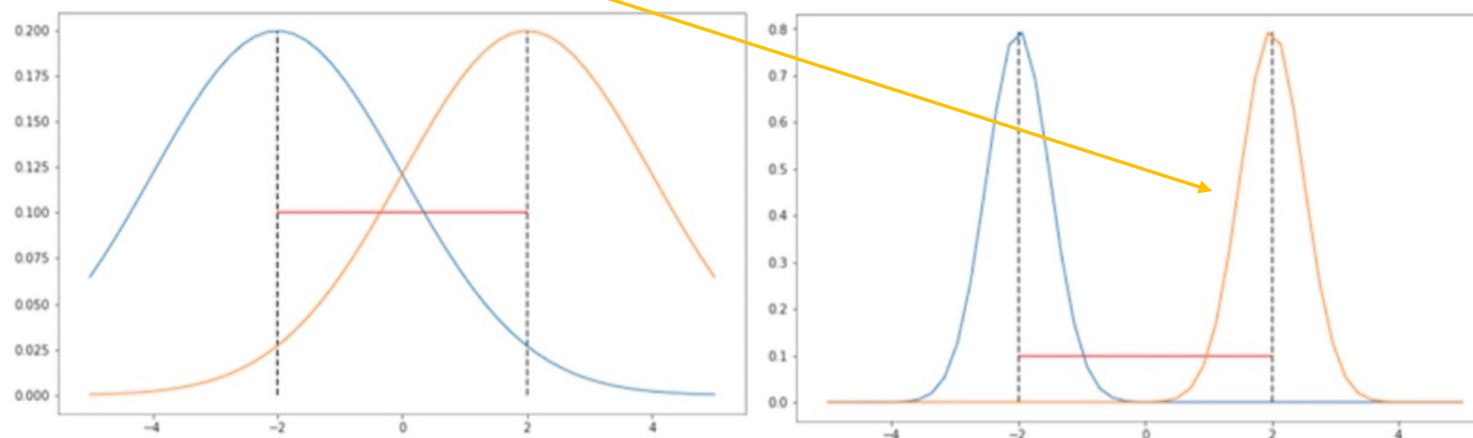
Is this typical steepest gradient descent correct?

Policy gradient sensitivity

In traditional neural networks, updates are performed within the **parameter space**. We assume that if the parameters θ_k change only slightly, the policy will also change only slightly. However, policy networks are highly **sensitive**.

For example, both updates shift the **mean** by the exact same amount (Euclidean distance = 4).

Right Figure: an action that was highly probable under the old policy has a probability of 0 under the new policy.
Bad for learning: 1) the **collapsed policy** distribution can be wrong; 2) no more exploration to correct the policy.



Catastrophic Collapse

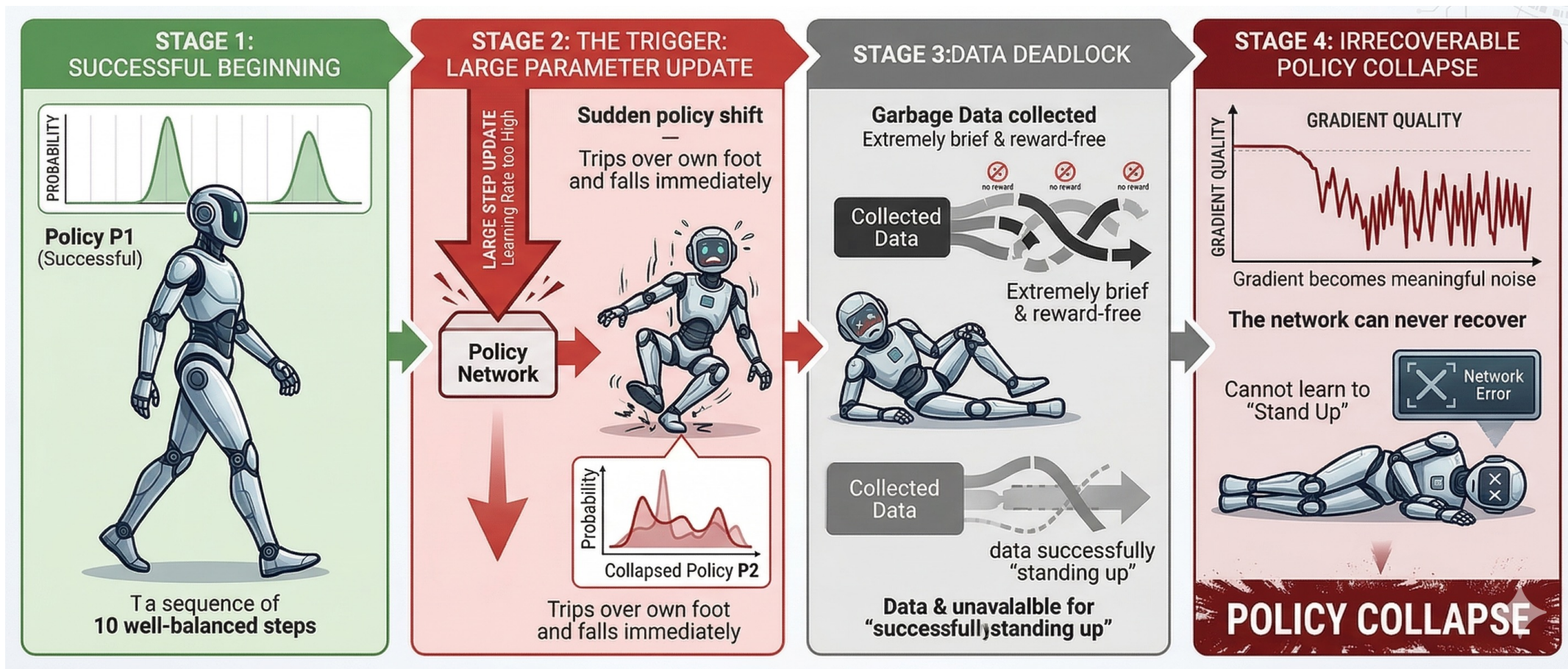


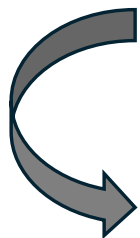
Image source: Gemini

Online data collected after a policy collapses will be biased.

Catastrophic Collapse

Policy updated
but collapsed
into something bad.

Sampling from such
a bad policy won't
provide useful
training data for LLM.



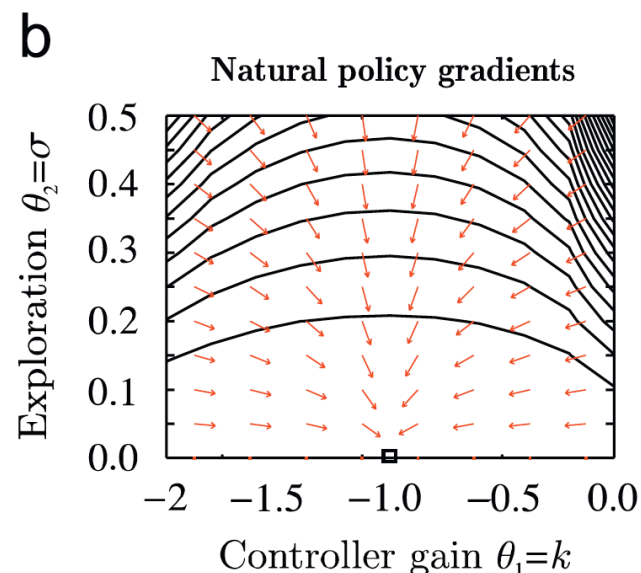
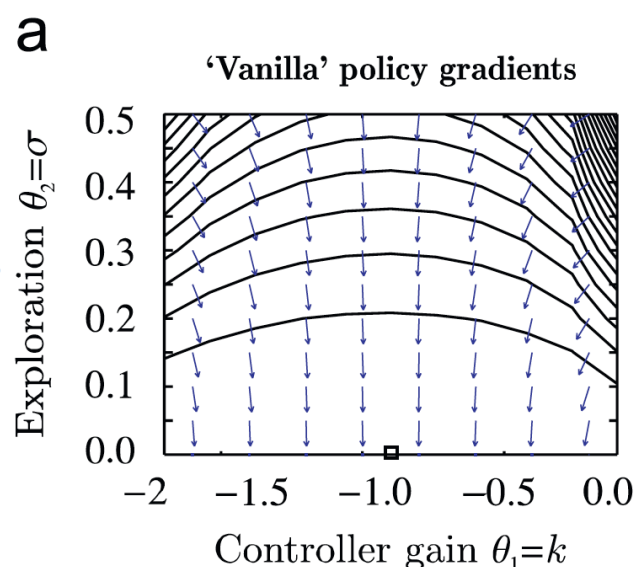
- The weather today is sunny and warm, perfect for a walk in the park. I think I'll bring a book and enjoy the breeze under the trees.
- The weather today is sunny and warm, perfect for a walk in the park. I think I'll bring a book and enjoy the breeze under the *trees trees trees the the the and and and perfect perfect perfect...*
- I love machine learning. *I love machine learning. I love machine learning. I love machine learning. I love machine learning...*

Image source: Gemini

Online data collected after a policy collapses will be biased.

Policy gradient direction issue

Following the vanilla policy gradients converges to some non-optimal solutions



Following the natural policy gradients converges to the optimal solution

Fig. 1. When plotting the expected return landscape for simple problem as 1d linear-quadratic regulation, the differences between (a) ‘vanilla’ and (b) natural policy gradients becomes apparent [21].

Image source: *Natural Actor-Critic*. Jan Petersa, Stefan Schaal. *Neurocomputing* 71 (2008) 1180–1190

KL Divergence

Measuring the magnitude of an update based on parameters change is undesirable.

Find a "*metric*" to directly quantifying "the difference between two policies' generating probability distributions", and use gradients in the space defined by that metric.

Solution: Replace the Euclidean norm constraint ($\|d\| \leq \epsilon$) with a **KL**

Divergence constraint $D_{KL}(P||Q) = \sum_x P(x) \log \left(\frac{P(x)}{Q(x)} \right)$

1. Non-negative: $D_{KL}(P||Q) \geq 0$ the KL divergence is 0 if and only if the two distributions are identical.

2. Asymmetric: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$ measures the "information loss when approximating P with Q", so it is a "divergence" rather than a strict distance.

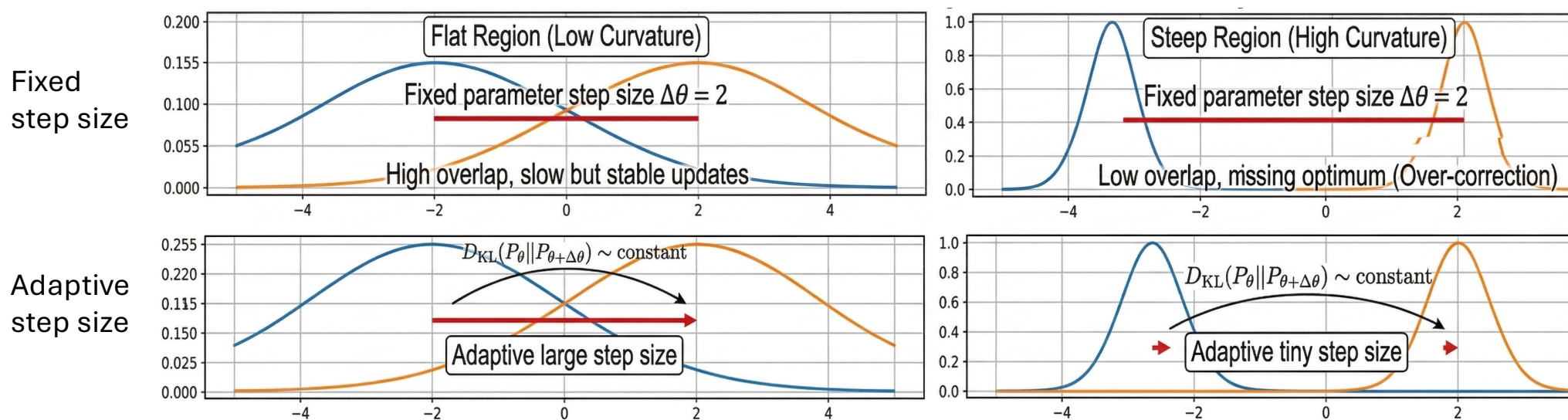
Why KL Divergence

Solution: replace the Euclidean norm constraint ($\|d\| \leq \epsilon$) with a **KL Divergence constraint**

$$(D_{KL} \leq \epsilon). \quad d^* = \arg \max_{d, \text{ s.t. } D_{KL}(\pi_\theta \| \pi_{\theta+d}) \leq \epsilon} V(\theta + d)$$

This ensures that the new policy is not too far away from the old policy (pre-trained policy), retaining basic linguistic capabilities while being aligned.

non-adaptive update can ruin pre-trained model's capabilities



Taylor Expansion of KL Divergence

Solving the exact KL constraint is intractable. We use **Taylor Expansion** to approximate:

$$d = \theta_{new} - \theta_{old} \quad D_{KL} \approx D_{KL}(\pi_{\theta} || \pi_{\theta}) + (\nabla_d D_{KL})^T d + \frac{1}{2} d^T (\nabla_d^2 D_{KL}) d$$

Expand $D_{KL}(\pi_{\theta_{old}} || \pi_{\theta_{old}+d})$ at $d = 0$:

0th-order term: $D_{KL}(\pi_{\theta} || \pi_{\theta}) = 0$ (KL divergence of the same distribution is 0)

First-order term: $(\nabla_d D_{KL})^T d = 0$ (the minimum value of the KL divergence is 0, the gradient at the minimum value must be a zero vector)

Second-order term: the Hessian matrix of the KL divergence is exactly equal to the Fisher Information Matrix (FIM) in statistics^[1], denoted as F .

Finally, we get: $D_{KL}(\pi_{\theta_{old}} || \pi_{\theta_{old}+d}) \approx \frac{1}{2} d^T F(\theta_{old}) d$

[1] <https://math.stackexchange.com/questions/2239040/show-that-fisher-information-matrix-is-the-second-order-gradient-of-kl-divergenc>

FIM Properties

$$D_{KL}(\pi_{\theta_{old}} \parallel \pi_{\theta_{old}+d}) \approx \frac{1}{2} d^\top F(\theta_{old}) d$$

Properties of Fisher Information Matrix (FIM)

1. Positive Semi-Definite (PSD): $F \succeq 0$ guarantees that $\frac{1}{2} d^\top F d$ will always be non-negative, perfectly matching the extensional property of the KL divergence $D_{KL} \geq 0$.

2. Symmetry: $F = F^\top$

F acts as a local metric tensor: it tells us how sensitive the distribution is to parameter changes. It is “local” because F depends on the current θ_{old} , and the sensitivity is different at different locations in the parameter space.

Natural Gradient

Solving the approximated constrained problem yields the **Natural Gradient** direction.

Maximize first-order approximation of the objective subject to second-order constraint over KL divergence.

$$\max_d \nabla_{\theta} V(\theta)^{\top} d \quad \text{s.t.} \quad \frac{1}{2} d^{\top} F d \leq \epsilon$$

Reformulate the constrained optimization problem as a Lagrangian form with $\lambda > 0$

$$\mathcal{L}(d, \lambda) = \nabla_{\theta} V(\theta)^{\top} d - \lambda \left(\frac{1}{2} d^{\top} F d - \epsilon \right)$$

Let the gradient of the Lagrangian be 0:

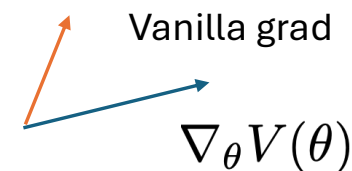
$$\nabla_d \mathcal{L}(d, \lambda) = \nabla_{\theta} V(\theta) - \lambda F d = 0 \quad \Rightarrow \quad d = \frac{1}{\lambda} F^{-1} \nabla_{\theta} V(\theta)$$

Natural grad

$$F^{-1} \nabla_{\theta} V(\theta)$$

Vanilla grad

$$\nabla_{\theta} V(\theta)$$



Natural Gradient

$$d = \frac{1}{\lambda} F^{-1} \nabla_{\theta} V(\theta) = \theta_{new} - \theta_{old}$$

Step Natural Gradient

$$\max_d \nabla_{\theta} V(\theta)^{\top} d$$

$$\text{s.t. } \frac{1}{2} d^{\top} F d \leq \epsilon$$

It can be abbreviated as $d = \alpha F^{-1} g$, solving for α

To maximize the objective function within the trust region, the optimal solution **must** lie on the boundary of the trust region. That is $\frac{1}{2} d^{\top} F d = \epsilon$. Otherwise, scale up d to satisfy the equality while increasing the objective.

Substitute $d = \alpha F^{-1} g$ into the constraint equation above: $\frac{1}{2} \alpha^2 (F^{-1} g)^{\top} F (F^{-1} g) = \epsilon$

Based on the property of matrix transpose $(AB)^{\top} = B^{\top} A^{\top}$, and the symmetry of Fisher's information matrix $(F^{-1})^{\top} = F^{-1}$: $\frac{1}{2} \alpha^2 (g^{\top} F^{-1}) F (F^{-1} g) = \frac{1}{2} \alpha^2 (g^{\top} F^{-1} g) = \epsilon$

Finally:

$$\alpha = \sqrt{\frac{2\epsilon}{g^{\top} F^{-1} g}}$$

Step-size is not a hyper-param

Natural Gradient Problems

Use k to represent iteration:

$$\theta_{k+1} = \theta_k + \underbrace{\sqrt{\frac{2\epsilon}{g^\top F^{-1}g}}}_{\text{step size}} \underbrace{F^{-1}g}_{\text{natural gradient}}$$

Problem 1: Matrix inversion is $\mathcal{O}(d^3)$

Dimension d could be the millions.

Solution 1: Using the **conjugate gradient method**, an iterative algorithm, we can approximate the result of the vector $F^{-1}g$ without actually calculating it.

This method is called truncated natural policy gradient (**TNPG**).

Truncated Natural Policy Gradient (TNPG)

Rephrase finding $v = F^{-1}g$ as solving the linear system $Fv = g$. We use **Conjugate Gradient (CG)** to solve this iteratively.

$\mathbf{r}_0 := \mathbf{b} - \mathbf{A}\mathbf{x}_0$

if $\mathbf{r}_0$ is sufficiently small, then return $\mathbf{x}_0$ as the result

$\mathbf{p}_0 := \mathbf{r}_0$

$k := 0$

repeat

$$\alpha_k := \frac{\mathbf{r}_k^\top \mathbf{r}_k}{\mathbf{p}_k^\top \mathbf{A} \mathbf{p}_k}$$

Update current solution.

$$\mathbf{x}_{k+1} := \mathbf{x}_k + \alpha_k \mathbf{p}_k$$

Update residual.

$$\mathbf{r}_{k+1} := \mathbf{r}_k - \alpha_k \mathbf{A} \mathbf{p}_k$$

if $\mathbf{r}_{k+1}$ is sufficiently small, then exit loop

Truncate if good enough.

$$\beta_k := \frac{\mathbf{r}_{k+1}^\top \mathbf{r}_{k+1}}{\mathbf{r}_k^\top \mathbf{r}_k}$$

Update direction.

$$\mathbf{p}_{k+1} := \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k$$

$$k := k + 1$$

end repeat

return $\mathbf{x}_{k+1}$ as the result

Don't need to evaluate the FIM (Hessian of KL divergence), since it is a large square matrix of $O(n^2)$, $n = |\theta|$.

First-order gradient vector is more feasible. Take the gradient of the inner product of gradient of KL divergence and v to approximate the Hessian-Vector Product:

$$\nabla^2 f(\theta) v = \lim_{\epsilon \rightarrow 0} \frac{1}{\epsilon} [\nabla f(\theta + \epsilon v) - \nabla f(\theta)] = \nabla [\langle \nabla f(\cdot), v \rangle](\theta) .$$

Source: https://en.wikipedia.org/wiki/Conjugate_gradient_method
<https://iclr-blogposts.github.io/2024/blog/bench-hvp/>

Natural Gradient Problems

Use g to replace $\nabla_{\theta} V(\theta)$ for concise.

$$\theta_{k+1} = \theta_k + \underbrace{\sqrt{\frac{2\epsilon}{g^{\top} F^{-1} g}}}_{\text{step size}} \underbrace{F^{-1} g}_{\text{natural gradient}}$$

Problem 2:

We derived the perfect step using Taylor expansion, but Taylor expansion is an approximation and contains errors. It is also expensive to evaluate due to inversion.

Solution 2:

Adding a safety measure: Adaptive step size, also known as line search.

TRPO

Solution 2: Adding a safety measure: Adaptive step size, also known as line search.

This method is called trust region policy optimization (**TRPO**)

$\mathcal{L}_{\theta_k}(\theta)$: The surrogate objective function. It measures the expected improvement of the new policy over the old policy. We strictly require $\mathcal{L}_{\theta_k}(\theta) > 0$ (must get better).

Compute NPG step Δ_k

for $j = 0, \dots, L$ **do**

 Compute update $\theta = \theta_k + \alpha^j \Delta_k$

if $\mathcal{L}_{\theta_k}(\theta) > 0$ and $\bar{D}_{KL}(\theta \parallel \theta_k) \leq \delta$ **then**

 Accept update and $\theta_{k+1} = \theta_k + \alpha^j \Delta_k$

break

end

endfor

- **Step 1 (Start Big):** Try the full Natural Gradient step ($j = 0, \alpha^0 = 1$).
- **Step 2 (Check Safety):** Does this step improve performance ($\mathcal{L} > 0$) AND satisfy the trust region ($D_{KL} \leq \delta$)?
- **Step 3 (Shrink & Retry):** If unsafe, exponentially shrink the step size by a factor of α (e.g., $1 \rightarrow 0.5 \rightarrow 0.25$) and test again until both conditions are met.

In practice, TRPO is implemented as TNPG + Line Search.

TRPO Problems

Although TRPO avoid computing $\mathcal{O}(n^3)$, but TRPO is still a **Second-Order Optimization** method. Besides, the CG algorithm, and thus the TRPO algorithm, still requires **multiple iterative backward passes** per update step. Optimizers like Adam only need to calculate the gradient once per step.

Can we safely abandon the second-order information?

- Second-order information due to natural policy gradient turns out to not so important.
- But the motivated usage of KL divergence to keep a safe update is useful.
 - KL divergence still models change of behavior of LLM policy, though don't use its second-order information.

[1] Schulman, John, et al. "Proximal policy optimization algorithms." arXiv preprint arXiv:1707.06347 (2017).

PPO

Proximal Policy Optimization (PPO):

Use the simplest and cheapest first-order standard descent (such as the Adam optimizer) but ensure safe update. First, we define:

$\pi(a|s)$ or $\pi_{\theta_{old}}$: Old Policy. The frozen network that interacted with the environment to collect the current batch of data.

$\pi'(a|s)$ or π_{θ} : New Policy. The active network whose parameters we are currently updating via gradient ascent.

PPO

The Probability Ratio: $r_{sa}(\pi') = \frac{\pi'(a|s)}{\pi(a|s)}$ (Note: this is not the reward)

$r_{sa} > 1$ Action a is more likely under s under the new policy.

$r_{sa} < 1$ Otherwise

The Advantage $A(s, a) = Q(s, a) - V(s)$: (Note: estimated using Monte Carlo rollouts)

$Q(s, a)$ (Action-Value): The expected return if we take a specific action a in state s .

$V(s)$ (State-Value / The Baseline): The average expected return of being in state s .

$A > 0$ The action was better than average (we want to encourage it).

$A < 0$ The action was worse than average (we want to discourage it).

Goal:

Maximize $r_{sa} \times A$: if advantage is high, increase probability of generating that action (token); otherwise, decrease that probability.

Make sure the update is safe using the KL divergence constraint.

PPO

The trust region via Clipping: $L^{CLIP}(\theta) = \hat{\mathbb{E}} \left[\min \left(r(\theta) \hat{A}, \text{clip}(r(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A} \right) \right]$

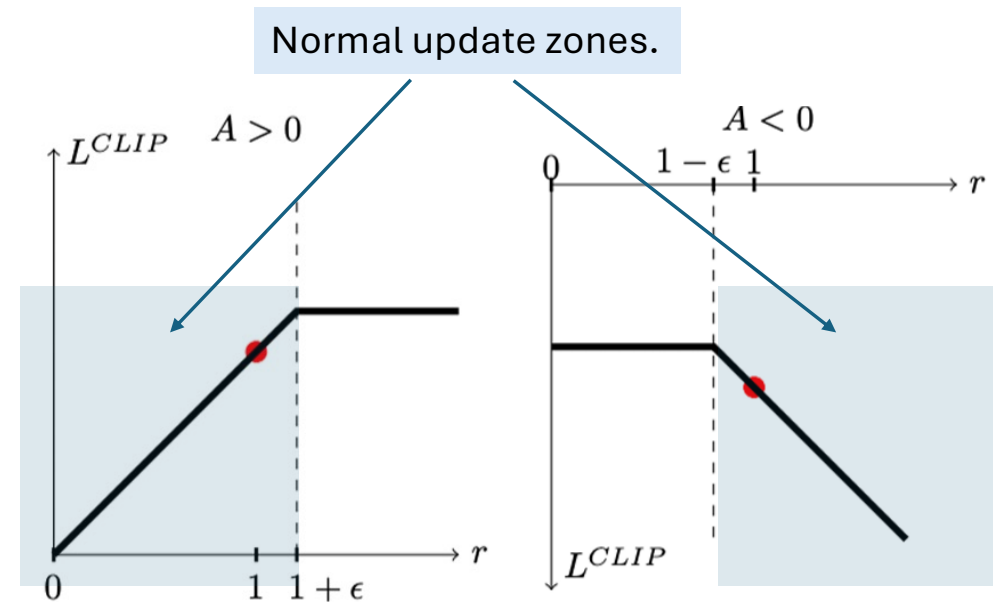
$$\text{clip}(r(\theta), 1 - \epsilon, 1 + \epsilon) = \begin{cases} 1 - \epsilon & \text{if } r(\theta) < 1 - \epsilon \\ r(\theta) & \text{if } 1 - \epsilon \leq r(\theta) \leq 1 + \epsilon \\ 1 + \epsilon & \text{if } r(\theta) > 1 + \epsilon \end{cases}$$

- **Case 1: Positive Advantage ($A > 0$)**

- The action is good; we want to increase the ratio.
- **The Clip:** Once r hits $1 + \epsilon$ (e.g., 1.2), the reward is capped.
- *Intuition:* "Good job, but don't be greedy. Stop updating once it's 20% more likely."

- **Case 2: Negative Advantage ($A < 0$)**

- The action is bad; we want to decrease the ratio.
- **The Clip:** Once r drops to $1 - \epsilon$ (e.g., 0.8), the penalty is capped.
- *Intuition:* "It's a bad action, but don't overreact. Decrease probability by 20% and observe."



| Extra readings

- For the trick of avoiding evaluating second-order Hessian in neural networks, see the article
 - <https://iclr-blogposts.github.io/2024/blog/bench-hvp/>
- For LLM optimization using RL, see the article by OpenAI
 - <https://spinningup.openai.com/en/latest/algorithms/ppo.html>

Quiz

1. Pen and paper; no compute or cellphone allowed.
 2. Turn in your answer sheet when you leave the classroom.
- Q1 (T/F): RLHF will keep updating the reward model during target LLM optimization.
 - Q2 (T/F): There are multiple forward-backward passes in per update of LLM parameters when using the trust-region policy optimization.
 - Q3 (multi-choice): KL divergence is used in RLHF because: (A) it helps reduce the number of LLM parameters; (B) it helps keeping the new policy close to the pretrained model; (C) it measures policy similarity better than just using policy parameter distance.
 - A: F; T; (BC)